MEAL-6 | Database Design & Implementation Guide

Meal Connect • Team Apex • PostgreSQL • Version 1.3 • 01 November 2026

Implementation reference based on the supplied MEAL-6 ER diagram, reviewed DBML and database model notes. The DBML defines a logical schema; executable migrations and service-layer rules are implemented during development.

1. Purpose and system boundary

Define the Meal Connect MVP data model, table relationships, important constraints, and traceability fields. Angular 21 communicates with FastAPI over REST/JSON; only FastAPI accesses PostgreSQL. The platform connects restaurants/bakeries offering surplus food, verified NGOs requesting quantities from existing donations, volunteers coordinating pickups, and admins verifying organizations.

MVP rule: a restaurant accepts the whole requested quantity or rejects the request. Partial approval is not supported. One donation may receive several NGO requests while available quantity remains.

2. Database entities and responsibilities

Table	Purpose
users	Platform identity, password hash, global role and account status
organizations	Restaurant / bakery / NGO profiles and current verification status
organization_memberships	Active membership and organization-scoped permissions
organization_verifications	Admin verification decision history
donations	Published food, safety information, deadlines and quantities
donation_requests	NGO quantity requests; full approve/reject decision
pickups	Collector assignment, pickup and delivery progression
distribution_records	Final NGO distribution, amount and beneficiaries
notifications	Recipient messages tied to system activity
status_history	Status transitions, actor, reason and timestamp

3. Core relationships and cardinality

Relationship	Multiplicity / rule
organizations → donations	1:N; one business publishes many donations
donations → donation_requests	1:N; one donation can receive many NGO requests
organizations (NGO) → donation_requests	1:N; NGO requests existing donations
users ↔ organizations	M:N through organization_memberships; (organization_id, user_id) unique

donation_requests → pickups	1:0..1; pickups.request_id UNIQUE and NOT NULL
pickups → distribution_records	1:0..1; distribution_records.pickup_id UNIQUE and NOT NULL
users → notifications	1:N; recipient_user_id foreign key
organizations → organization_verifications	1:N; each admin decision logged

4. Key columns and traceability

Area	Required fields / behavior
Record identity	BIGINT primary keys (e.g., donation_id, request_id, pickup_id) with foreign-key ownership links
Actors	created_by_user_id, requested_by_user_id, reviewed_by_user_id, collector_user_id, recorded_by_user_id, changed_by_user_id
Status / decisions	status, decision, old_status, new_status, decision_note, change_reason, review_note
Audit timestamps	created_at / updated_at, reviewed_at, verified_at, joined_at, changed_at; UTC-aware timestamptz
Food lifecycle	prepared_at, available_from, pickup_deadline, scheduled_at, picked_up_at, delivered_at, distributed_at
Quantity trace	donations.total_quantity, donations.allocated_quantity, donation_requests.requested_quantity, distribution_records.distributed_quantity
Polymorphic references	notifications.related_entity_type + related_entity_id; status_history.entity_type + entity_id; validate targets in FastAPI

5. Database constraints

- UNIQUE: users.email; organizations.registration_number when provided; membership (organization_id, user_id); pickups.request_id; distribution_records.pickup_id.
- FOREIGN KEYS: enforce ordinary ownership and user references. Polymorphic entity_type/entity_id cannot use a conventional single-target foreign key.
- CHECK: total_quantity > 0, 0 <= allocated_quantity <= total_quantity, requested_quantity > 0, distributed_quantity > 0, beneficiary_count >= 0.
- TIME: pickup_deadline > available_from; prepared_at (if present) <= pickup_deadline; pickup timestamp sequence must be coherent.
- EMAIL: normalize to lowercase (or implement case-insensitive uniqueness). Store a password hash, never plaintext.
- UNIT: requests and final distributions use the same unit as the parent donation; enforce compatibility in FastAPI.
- ROLE: users.role controls global capability; organization_memberships.membership_role plus active member_status controls each organization action.

- VERIFICATION: only approved restaurants/bakeries publish; only approved NGOs request. Enforce in FastAPI; a foreign key alone is insufficient.

6. Transaction-safe donation approval

Available quantity = total_quantity − allocated_quantity. An NGO request starts as pending and must not reserve food until approved. Approval must be atomic; merely displaying a quantity in the frontend is not a guarantee.

- Start a single database transaction; lock the pending request and its donation row in a consistent order.
- Validate organization permission, donation status, deadline, and remaining available quantity.
- Conditionally increment allocated_quantity by the entire requested_quantity; reject if capacity is insufficient.
- Update the request from pending to approved in the same transaction and insert status_history.
- Verify row counts; if any check fails, roll back all changes. Commit only once all operations succeed.
- On cancellation before pickup, release the reserved quantity exactly once in a transaction; never automatically release after collection.

Illustrative PostgreSQL migration checks

The following SQL enforces single-row quantity limits; cross-table policies remain FastAPI responsibilities:

```
ALTER TABLE donations ADD CONSTRAINT ck_donation_positive_total CHECK (total_quantity > 0);
ALTER TABLE donations ADD CONSTRAINT ck_donation_allocation_bounds
  CHECK (allocated_quantity >= 0 AND allocated_quantity <= total_quantity);
ALTER TABLE donations ADD CONSTRAINT ck_donation_deadline
  CHECK (pickup_deadline > available_from);
ALTER TABLE donation_requests ADD CONSTRAINT ck_request_quantity
  CHECK (requested_quantity > 0);
ALTER TABLE distribution_records ADD CONSTRAINT ck_distribution_quantity
  CHECK (distributed_quantity > 0);
ALTER TABLE distribution_records ADD CONSTRAINT ck_beneficiary_count
  CHECK (beneficiary_count >= 0);
```

7. Lifecycle and business enforcement

Action	Required check
Publish donation	Approved business organization, permitted active membership, valid quantity and deadline
NGO requests quantity	Approved NGO, active membership, valid donation and positive request quantity
Approve / reject	Only donating organization; full quantity only; no approved_quantity field
Create pickup	Request already approved; pickup request_id unique; collector must be authorized volunteer
Record distribution	Successful pickup, NGO authorization, quantity > 0 and <= approved requested quantity
Cancel after approval	Before collection: reverse allocation once; after collection: do not release without reversal process
Expiry / status changes	Reject new requests for expired/cancelled listings; log actor, timestamp, previous/new status

8. Data flow and implementation order

- Create PostgreSQL schema using migrations derived from reviewed DBML; build enums, tables, PK/FK, UNIQUE constraints, CHECK constraints and indexes.
- Implement user and organization authentication, membership authorization and verification gates in FastAPI.
- Implement donation create/list endpoints and NGO request creation from existing donation IDs.
- Implement transaction-safe full approve/reject, cancellation and status_history recording.
- Implement pickup, distribution and notification logic; generate reports from confirmed stored records.
- Expose detailed REST payloads and error contracts in MEAL-62, not duplicated in this database guide.
- Test concurrent approvals, repeat cancellation, invalid cross-organization access, duplicate pickup/distribution and incorrect status transitions.

9. Source artifacts and important distinction

Source artifacts: Meal_Connect_MEAL-6_Final_ER_Diagram.pdf, Meal_Connect_MEAL-6_Reviewed.dbml, and Meal_Connect_MEAL-6_Reviewed_Notes.md. The diagram and DBML are authoritative descriptions of the proposed logical model, not proof that PostgreSQL constraints are deployed.

This guide focuses on implementation and deliberately contains no review, approval or sign-off sections. Detailed API routes and request/response contracts belong to MEAL-62.